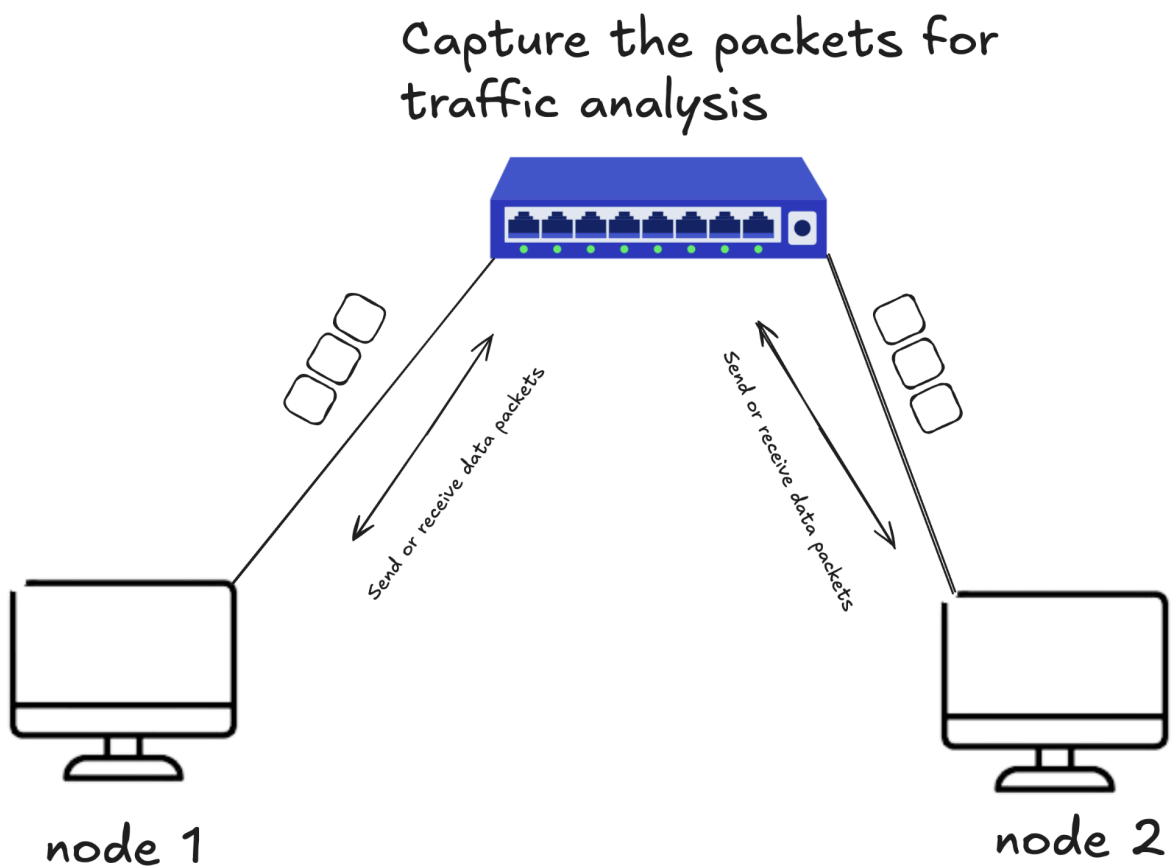


NAME: KAVYA JAIN

CPP PROJECT SIMULATION

AIM: TO SIMULATE THIS NETWORK



Network Simulation Approaches Considered

This section summarizes the different approaches considered for creating a network environment for traffic generation, packet capture, and analysis. Each approach was evaluated based on realism, scalability, control over network parameters, and suitability for reproducible experiments.

1. Virtual Machines

Creating separate virtual machines to represent client, server, and analyzer nodes.

Limitations:

- High resource consumption in terms of CPU and memory.
- Slow startup and management overhead.
- Scaling to larger network topologies becomes inefficient.
- Managing networking between many VMs becomes complex.
- onfiguration using external tools to emulate realistic network behavior.

2. Capturing Real Internet Traffic

Collecting packet traces from real-world applications such as video streaming or gaming on a normal operating system.

Limitations:

- Traffic traces contain background traffic such as DNS queries, operating system updates, and unrelated application flows.
- Difficult to isolate specific application traffic.
- Lack of reliable ground truth labels for classification.
- Experiments become difficult to reproduce.

3. Docker-Based Network Emulation

Extending Docker networking with additional tools to simulate network behavior.

Limitations:

- Requires combining multiple tools and configurations.
- Not specifically designed for networking research.

- More complex to implement advanced network experiments.

Selected Approach: Mininet

Mininet was selected as the primary environment for the network simulation.

Reasons for selection:

- Lightweight architecture based on Linux network namespaces.
- Ability to create virtual hosts, switches, and links easily.
- Built-in support for simulating bandwidth, latency, and packet loss.
- Runs real applications and uses the actual Linux networking stack.
- Enables reproducible and scalable experiments.
- Widely used in networking research and experimental studies.

Mininet therefore provides an effective balance between realism, flexibility, and efficiency for implementing the required traffic generation, packet capture, and analysis pipeline.

How Mininet Works Internally

Mininet is a lightweight network emulator that allows the creation of realistic virtual networks on a single machine. It achieves this by leveraging core Linux kernel networking features. Instead of simulating network behavior abstractly, Mininet runs real networking code and real applications while isolating components using Linux primitives.

Mininet primarily relies on three fundamental mechanisms within the Linux operating system:

1. Network Namespaces
2. Virtual Ethernet (veth) Pairs
3. Virtual Switches

1. Network Namespaces

Network namespaces provide isolation for network resources within the Linux kernel. Each namespace maintains its own independent network stack.

This means that every Mininet host has:

- A separate IP address space

- Its own routing table
- Independent network interfaces
- Separate ARP tables and network configuration

As a result, each Mininet host behaves like a completely independent machine even though all hosts are running on the same physical system.

Example:

Host h1

IP Address: 10.0.0.1

Host h2

IP Address: 10.0.0.2

Although both hosts exist on the same physical machine, the network namespace mechanism ensures that they operate as if they were separate devices connected over a network.

2. Virtual Ethernet (veth) Pairs

Mininet connects hosts and switches using virtual Ethernet interfaces known as veth pairs.

A veth pair functions similarly to a physical network cable. It consists of two interconnected virtual interfaces. Any packet transmitted on one end of the pair immediately appears on the other end.

Conceptually:

veth0 <---- virtual cable ----> veth1

One interface of the pair is placed inside a host namespace, while the other is attached to a switch.

For example:

- h1-eth0 connects the host h1 to the switch
- s1-eth1 represents the switch-side interface

This mechanism allows packets to travel between hosts and switches exactly as they would in a physical network.

3. Virtual Switch

Mininet typically uses Open vSwitch to emulate network switches. Open vSwitch is a production-quality multilayer virtual switch widely used in cloud and data center environments.

The switch receives packets from one interface and forwards them to the appropriate destination interface based on forwarding rules, similar to a physical Ethernet switch.

Example topology:

h1-eth0 ---- s1 ---- h2-eth0

In this topology:

- h1 sends packets through interface h1-eth0
- The packets arrive at the switch s1
- The switch forwards the packets to h2 via interface h2-eth0

The packet forwarding process follows the same logic as in real-world networks, enabling realistic experimentation and testing.

Summary

By combining network namespaces, virtual Ethernet pairs, and virtual switches, Mininet creates complete virtual network topologies on a single machine. These mechanisms allow researchers and developers to run real network protocols, generate traffic, capture packets, and evaluate network behavior in a controlled and reproducible environment.

This architecture makes Mininet particularly useful for networking research, protocol testing, traffic analysis, and quality-of-service experimentation.

```
client namespace
|
veth-client
|
| (virtual cable)
|
veth-client-br
  \
   \ br0 (virtual switch)
    /
   /
veth-server-br
|
|
veth-server
|
server namespace
```

INSTALL MINIMAP:

sudo apt install mininet

1. topology.py

```
from mininet.net import Mininet
from mininet.topo import Topo
from mininet.cli import CLI

class QUICtopo(Topo):
    def build(self):
        client = self.addHost('h1')
        server = self.addHost('h2')
        analyzer = self.addHost('h3')
        switch = self.addSwitch('s1')

        self.addLink(client, switch)
        self.addLink(server, switch)
        self.addLink(analyzer, switch)

topo = QUICtopo()
net = Mininet(topo)
net.start()

CLI(net)
net.stop()
```

2. `sudo python3 topology.py`

3. Start QUIC Server

mininet> h2 bash

`python -m aioquic.examples.http3_server --port 4433`

GENERATE QUIC TRAFFIC USING : *aioquic*

```
import asyncio
from aioquic.asyncio.client import connect
from aioquic.quic.configuration import QuicConfiguration

SERVER = "10.0.0.2"
PORT = 4433
REQUESTS = 50

async def run():
    configuration = QuicConfiguration(is_client=True)

    for i in range(REQUESTS):
        print("Sending request", i+1)

        async with connect(SERVER, PORT, configuration=configuration) as client:
            stream_id = client._quic.get_next_available_stream_id()
            client._quic.send_stream_data(stream_id, b"hello", end_stream=True)
            await asyncio.sleep(0.1)

asyncio.run(run())
```

4. CAPTURE PACKETS:

`sudo tcpdump -i s1-eth1 udp -w quic_capture.pcap`

5. ANALYZE USING *scapy.rdcap*

```

from scapy.all import rdpcap
import pandas as pd
import numpy as np

packets = rdpcap("quic.pcap")

records = []

for pkt in packets:
    if pkt.haslayer("IP") and pkt.haslayer("UDP"):
        records.append({
            "time": pkt.time,
            "src_ip": pkt["IP"].src,
            "dst_ip": pkt["IP"].dst,
            "src_port": pkt["UDP"].sport,
            "dst_port": pkt["UDP"].dport,
            "packet_size": len(pkt)
        })

df = pd.DataFrame(records)

# -----
# Flow identification
# -----

df["flow_id"] = (
    df["src_ip"] + "_" +
    df["dst_ip"] + "_" +
    df["src_port"].astype(str) + "_" +
    df["dst_port"].astype(str)

```

)

Packet level features

```
packet_features = {  
    "total_packets": len(df),  
    "avg_packet_size": df["packet_size"].mean(),  
    "min_packet_size": df["packet_size"].min(),  
    "max_packet_size": df["packet_size"].max(),  
    "std_packet_size": df["packet_size"].std()  
}
```

Timing features

```
df["inter_arrival"] = df["time"].diff()
```

```
timing_features = {  
    "avg_interarrival": df["inter_arrival"].mean(),  
    "min_interarrival": df["inter_arrival"].min(),  
    "max_interarrival": df["inter_arrival"].max(),  
    "std_interarrival": df["inter_arrival"].std()  
}
```

Flow duration

```
flow_duration = df["time"].max() - df["time"].min()
```

```
flow_features = {  
    "flow_duration": flow_duration,  
    "packet_rate": len(df) / flow_duration if flow_duration > 0  
else 0,  
    "byte_rate": df["packet_size"].sum() / flow_duration if  
flow_duration > 0 else 0  
}
```

```
# -----  
# Directional features  
# -----
```

```
src_ip = df.iloc[0]["src_ip"]
```

```
forward = df[df["src_ip"] == src_ip]  
backward = df[df["src_ip"] != src_ip]
```

```
direction_features = {  
    "forward_packets": len(forward),  
    "backward_packets": len(backward),  
    "forward_bytes": forward["packet_size"].sum(),  
    "backward_bytes": backward["packet_size"].sum(),  
}
```

```
# -----  
# Burst features  
# -----
```

```
burst_threshold = 0.01
```

```
bursts = df["inter_arrival"] < burst_threshold
```

```
burst_features = {  
    "burst_packets": bursts.sum(),  
    "burst_ratio": bursts.sum() / len(df)  
}
```

```
# -----  
# Combine all features  
# -----
```

```
features = {}  
features.update(packet_features)  
features.update(timing_features)  
features.update(flow_features)  
features.update(direction_features)  
features.update(burst_features)
```

```
feature_df = pd.DataFrame([features])
```

```
print("\nExtracted Features:\n")  
print(feature_df)
```